

Hajee Mohammad Danesh Science and Technology University
MSc in CSE

Dept. of Computer Science

Course: Software Project Management

Course Code: CSE 531

Prepared by: Professor Dr. Ashis Kumar Mandal

Reference Books: 1. Introduction to Software Project Management by Adolfo Villafiorita
 2. Software Engineering by Ian Sommerville

Making IT Better: Managing Changes, Risks, and Quality

Change Management in Software Project Management

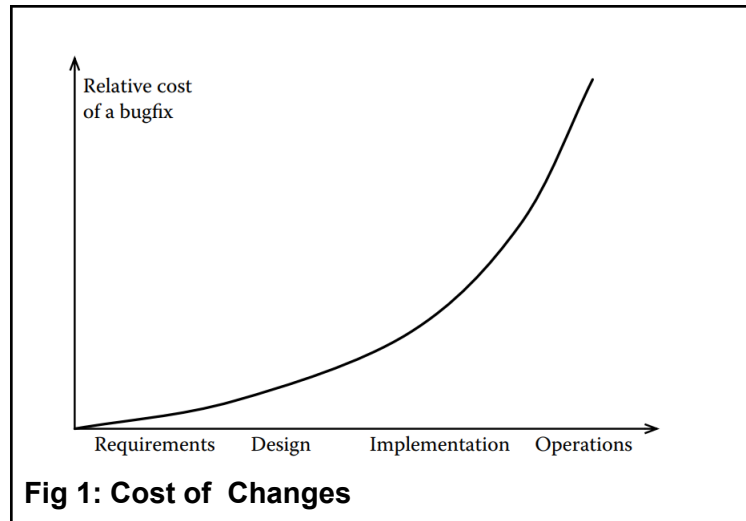
Change management in software project management refers to the structured approach of handling any changes or modifications to a project throughout its lifecycle. This includes changes in requirements, technology, resources, or even scope.

Managing Changes

Throughout a project, both internal and external stakeholders may request changes to the work being performed or the characteristics of the deliverables. These requests can arise for a variety of reasons, including:

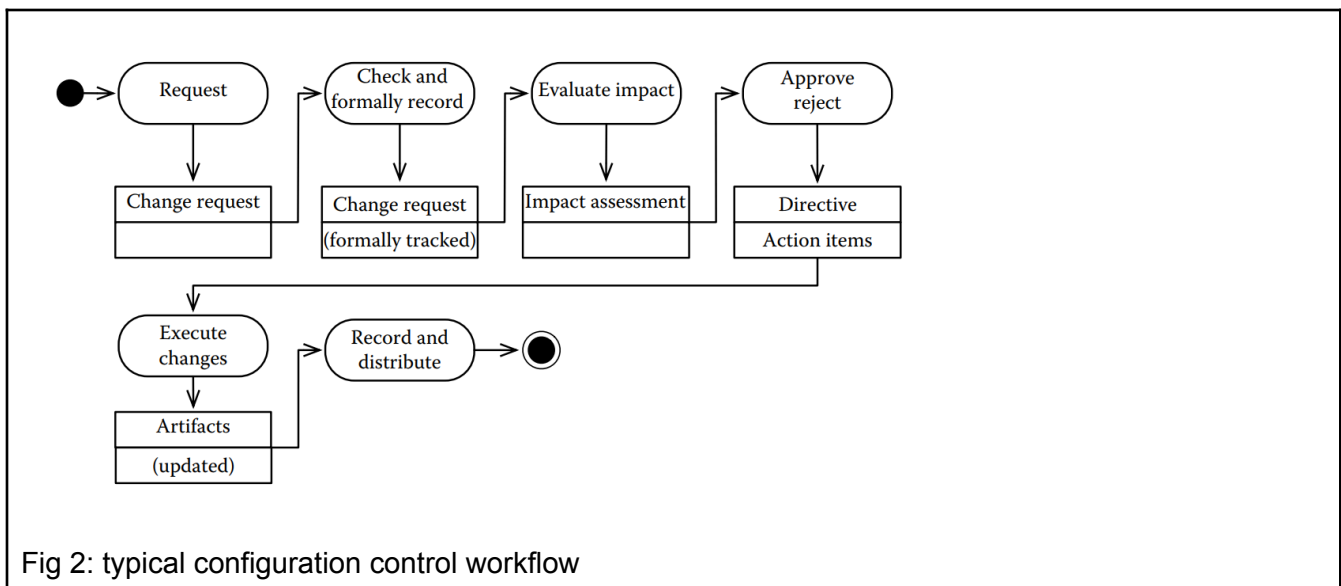
- **Incomplete or Inconsistent Requirements:** Issues with project requirements or work descriptions that were not initially apparent at the start of the project.
- **Improved Understanding:** A deeper understanding of the system being developed, presenting an opportunity to construct the deliverable more effectively.
- **Technical Opportunities:** The discovery of a more efficient or feature-rich approach that could enhance the deliverable.
- **Technical Challenges:** Unforeseen obstacles that make it impossible to complete the deliverable with the originally chosen approach.
- **Changes in the External Environment:** Shifts such as changes in stakeholder opinions or external factors, like the launch of a new competitive product.
- **Noncompliance:** When a deliverable fails to meet its original specifications.

Figure 1 shows the increasing cost of fixing a bug during different phases of software development.



Various processes have been proposed to manage changes, and they all share four key characteristics:

1. **Formal Documentation:** They ensure that any request for a change is formally recorded and documented.
2. **Decision Criteria:** They define the process for determining whether a change request will be accepted or rejected.
3. **Lifecycle Tracking:** They track the entire lifecycle of change requests, enabling confirmation of whether a request was approved and, if so, when it occurred.
4. **Stakeholder Involvement:** They specify the individuals or groups who need to be involved in or informed about the change.



The configuration control workflow manages and tracks changes systematically.

1. **Request:** A change is proposed and documented.
2. **Record:** The request is reviewed and formally logged.
3. **Impact Evaluation:** The change's impact is analyzed.
4. **Decision:** The change is approved or rejected.
5. **Execution:** Approved changes are implemented, updating relevant artifacts.
6. **Distribution:** Updated documents and artifacts are recorded and shared.

Configuration Management (CM) is a systematic process to ensure that all software artifacts (e.g., source code, documentation, scripts) are well-organized, tracked, and controlled throughout the software lifecycle. It ensures that changes are managed efficiently and the system remains consistent over time.

Software Artifacts: Includes source code, requirements, architecture documents, test plans, test outputs, and manuals. These are the building blocks of the system.

Challenges in CM:

- **Impact of Changes:** A small change (e.g., in a requirement) can affect multiple artifacts (e.g., source code, test cases, user manuals).
- **Multiple Configurations:** Systems exist in various states (e.g., versions for different users or platforms), requiring support for all versions in use.

Key Activities in CM

1. **Configuration Identification:**
 - Define all items in the system (source code, documents, scripts, etc.).
 - Specify item characteristics and relationships.
 - Use version numbers and baselines for tracking.
2. **Configuration Status Accounting:**
 - Track the history of changes and the status of configuration items.
 - Keep formal records of item characteristics and baseline versions.
3. **Release Management:**
 - Control the release process (e.g., deploying the software to production).
 - Include necessary artifacts like release notes detailing changes and updates.
4. **Data Archival and Accessibility:**
 - Ensure all artifacts are stored securely and remain accessible for future use.
 - Prevent scenarios where important files are lost or only stored locally.

Versioning Systems and Software Evolution Models

Versioning systems are tools that manage the evolution of software systems by tracking changes, managing multiple versions, and facilitating collaboration. Software evolves using either a linear model or a branching model.

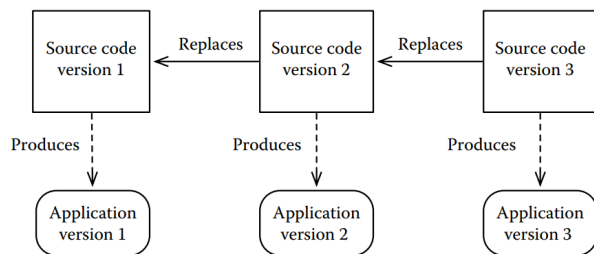
Software Evolution Models

1. Linear Model:

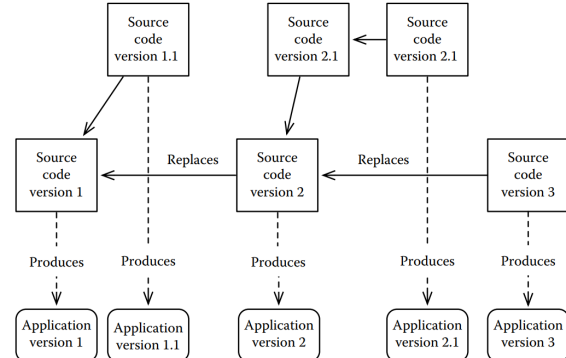
- Software evolves in a straight line, replacing old versions with new ones.
- Suitable for single-version applications like web apps (e.g., Google Docs).
- Rarely requires older versions unless reverting due to a bug.

2. Branching Model:

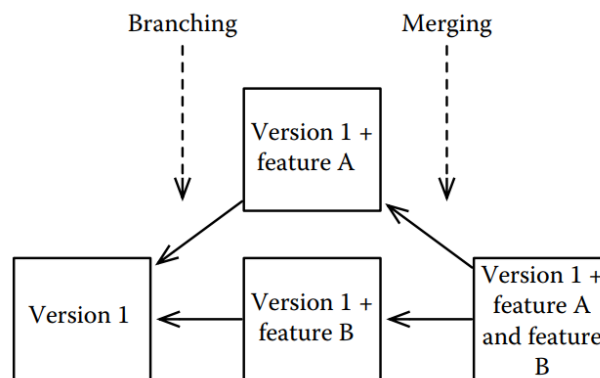
- Used when multiple configurations or versions of software exist (e.g., Internet Explorer versions).
- Different versions evolve independently on separate branches, represented as a tree or graph.
- **Merging:** Branches can be combined to create a new version with features from both branches.



The linear development model.



The branching evolution model



A merge of software branches.

Key Features of Versioning Systems

1. Repository:

- Stores all versions of the system.
 - Can be **centralized** (single repository) or **distributed** (multiple repositories, popular in open source).
2. **Working Copy:**
- Local copy used by developers for changes.
 - Developers use commands like:
 - **Commit:** Save changes to the repository.
 - **Branch:** Create a new branch for parallel work.
 - **Merge:** Combine branches with a common ancestor.
3. **Branching and Merging:**
- New branches are created for features or fixes.
 - Merging integrates changes from different branches into a single version.

Version Numbering System

- Versions are identified using a format like **N.M.P**, where:
 - **P:** Incremented for minor changes (e.g., bug fixes).
 - **M:** Incremented for significant updates (e.g., adding features).
 - **N:** Changed for major modifications (e.g., redesigns).
- Additional labels:
 - **Alpha:** Early development stage.
 - **Beta:** Pre-release for testing.
 - **RCX:** Release candidate (near-final version).

Example:

- Major changes: Moving from version 1.0.0 to 2.0.0.
- Minor: Moving from version 1.0.0 to 1.1.0 indicates added features while maintaining the same overall system.
- Patch version P : Moving from version 1.1.0 to 1.1.1 fixes a bug in version 1.1.0.

Importance of Versioning Systems

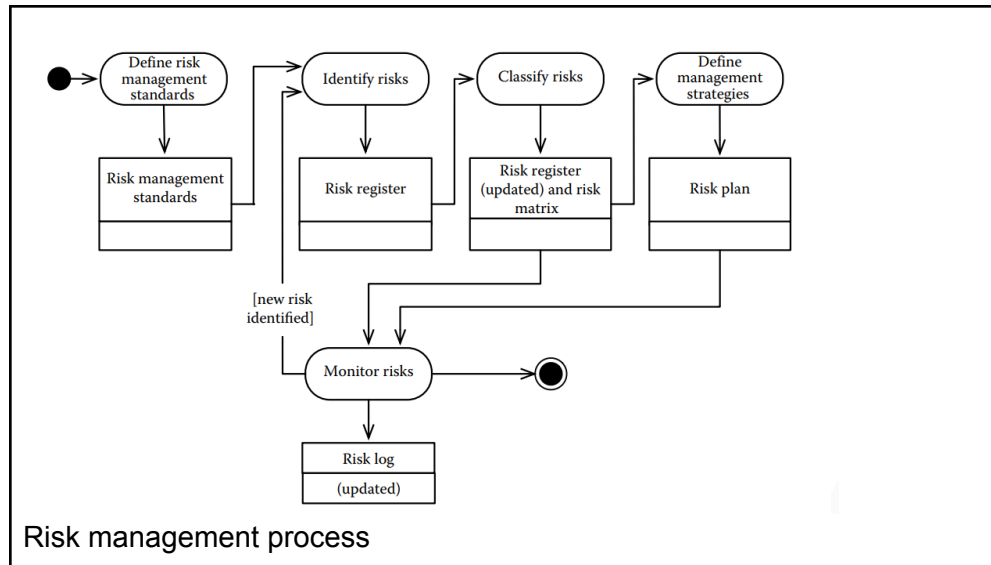
- **Track Changes:** Maintains a history of edits and developments.
- **Parallel Development:** Allows multiple developers to work independently.
- **Revertibility:** Restores previous versions if issues arise.
- **Efficiency:** Separates concerns using branches and merges them when ready.

Versioning is crucial for managing complex, evolving software systems effectively.

Risk Management

We use that of Project Management Institute (2004), where a risk is defined as “an uncertain event or condition that, if it occurs, has a positive or a negative effect on a project objective. A risk has a cause and, if it occurs, a consequence.

Risk Management: The process of identifying, analyzing, prioritizing, and responding to risks throughout the project lifecycle.



Risk management process:

- **Define Risk Management Standards:** In this phase, the project manager establishes the process for identifying risks and defines the procedures for managing them when they arise.
- **Identify Risks:** This step involves identifying potential risks that may affect the project.
- **Classify Risks:** In this phase, the identified risks are classified by assigning probabilities and impacts. This classification helps prioritize which risks should be closely monitored. Various techniques, both qualitative and quantitative, can be used for risk classification.
- **Develop a Management Strategy:** Based on the classification of risks in the previous step, a strategy is formulated for each risk or group of similar risks. This ensures that the project manager and stakeholders can agree on the policies to follow if a risk materializes.
- **Monitor Risks:** During project execution, risks are continuously monitored, and the appropriate strategies are implemented if any risk occurs.

Common risks in software development projects: Barry Boehm's famous list of risks identifies the following key reasons for project failure:

1. **Personnel Shortfalls:** Difficulty in finding qualified personnel.
2. **Unrealistic Schedules and Budgets:** Challenges in setting feasible timelines and costs.

3. **Incorrect Requirements:** Developing the wrong features or user interface due to poor requirement understanding.
4. **Gold-Plating:** Developing unnecessary features due to client requests or team preferences.
5. **Scope Creep:** Continuous requirement changes.
6. **Subcontractor Issues:** Poor quality of externally performed tasks.
7. **Performance Shortfalls:** Technical difficulties in building a complex system.

Other sources highlight risks like **lack of stakeholder involvement**, **poor change management**, and **inadequate project management**. Sommerville classifies risks into three areas: project-related risks (e.g., schedule, costs), product-related risks (e.g., performance, quality), and business-related risks (e.g., marketability).

Classify Risks

Risks are identified, they are classified based on two factors:

1. **Probability:** The likelihood of the risk occurring.
2. **Impact:** The potential consequences if the risk occurs.

The combination of probability and impact helps determine which risks should be monitored closely. There are two main methods to assess these factors:

- **Qualitative Approach:** A subjective evaluation of the probability and impact, typically using a predefined scale. A common scale uses five levels to classify both probability and impact. This provides a qualitative measure of how likely a risk is and how severe its impact could be.

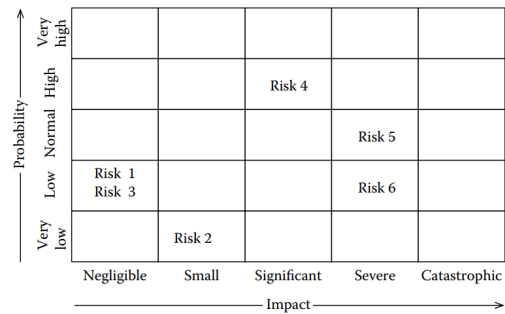
Table 4.1, where both probability and impact are classified with one out of five values.

First approach:

Risks are visualized on a two-dimensional **risk matrix**, where one axis represents probability and the other represents impact. Each risk is placed in the matrix based on its probability and impact values. For example, "Risk 1" and "Risk 3" have low probability and very low impact. This matrix helps categorize risks for easier monitoring and management.

Table 4.1 Common Probability and Impact Ranges

<i>Probability</i>			<i>Impact</i>		
<i>As Text</i>	<i>As a Real Number</i>	<i>As an Integer</i>	<i>As Text</i>	<i>As a Real Number</i>	<i>As an Integer</i>
Very low	0.2	1	Negligible	0.2	1
Low	0.4	2	Small	0.4	2
Normal	0.6	3	Significant	0.6	3
High	0.8	4	Severe	0.8	4
Very high	1	5	Catastrophic	1	5

**Figure 4.7 A risk matrix highlighting menaces.****Table 4.2 Classification of Menaces Using a Risk Register**

<i>Risk Description</i>	<i>Probability</i>	<i>Impact</i>	<i>Weight</i>
Risk 1	2	1	2
Risk 2	1	2	2
Risk 3	2	1	2
Risk 4	4	3	12
Risk 5	3	4	12
Risk 6	2	3	6

Second approach:

In the second approach, the risks are assigned a weight, computed as a function of the probability and impact. The main requirement of the function is that it is monotonic with respect to both probability and impact (e.g., it gives higher values when probability or impact increase). A very simple function to compute the weight is

$$\text{Weight} = \text{probability} \times \text{impact}$$

Risk with probability 3 and impact 5 has the same weight and importance of a risk with probability 5 and impact 3. For instance, in Table 4.2, where we list the same risks of Figure 4.7. Notice how “Risk 4” and “Risk 5” have the same weight, in spite of belonging to two different cells in the risk matrix of Figure 4.7.

Risk management strategies: focus on reducing the probability or impact of risks. These strategies include:

1. **Avoidance:** Adjusting the project plan to prevent the risk from occurring, which is the most cautious approach, but may not be feasible for all risks.

2. **Mitigation:** Modifying the project plan to reduce the probability or impact of the risk. Avoidance is a specific form of mitigation where the risk is entirely eliminated.
3. **Transferral:** Shifting the risk to another party, such as through insurance, to manage the impact.

Opportunities are **positive risks**—things that could benefit the project. You can manage them in different ways:

1. Exploitation
 - Make sure the opportunity definitely happens.
 - Example: Fast-tracking a new technology to gain a competitive advantage.
2. Enhancement
 - Increase the chance or benefit of the opportunity.
 - Example: Training your team to be more productive so the opportunity is more likely to succeed.
3. Sharing
 - Work with others (partners, stakeholders) to improve the outcome.
 - Example: Collaborating with another company to expand market reach.
4. Acceptance
 - Recognize the opportunity but don't change the plan proactively.
 - Take advantage only if it occurs naturally.
 - Example: Waiting to see if a government grant becomes available before acting.

Risks are often classified into three zones based on their severity:

- **Green zone:** Risks considered acceptable.
- **Yellow zone:** Risks requiring monitoring and special management.
- **Red zone:** Risks that are unacceptable and need to be avoided.

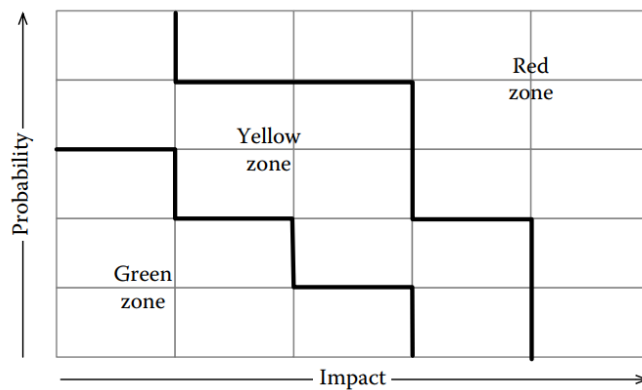


Figure 4.8 Risk management strategies.

Budgeting for Risks

Projects should allocate a specific budget for managing risks, which is added to the overall project budget. There are three approaches to defining a risk budget:

1. **Lump sum allocation:** A percentage of the project budget, often based on historical data, can be allocated for risks (up to 30% in some cases).
2. **Probability and impact-based allocation:** The budget is assigned based on the likelihood and severity of risks:
 - High-probability, low-impact risks should be fully budgeted.
 - Low-probability, low-impact risks can use expected monetary value, calculated by multiplying the risk's impact by its probability.
 - Low-probability, high-impact risks require special agreements like insurance rather than including them directly in the project budget.

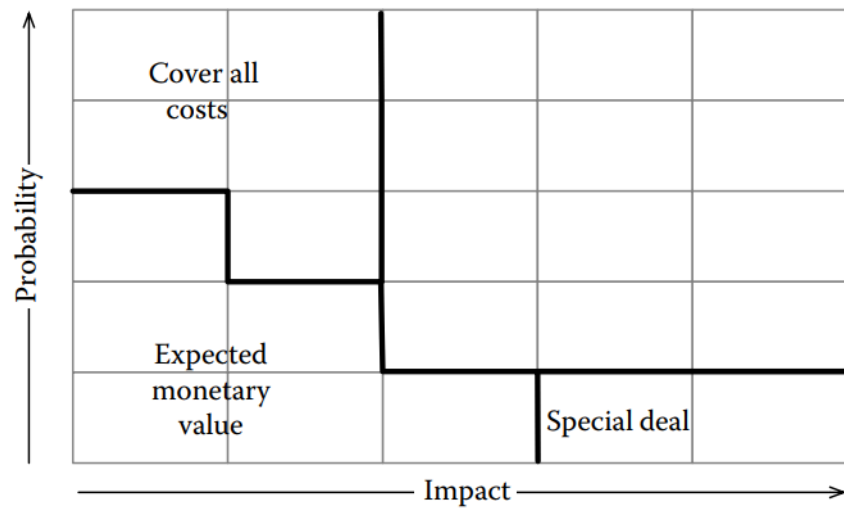


Figure 4.9 Budgeting for risks.

Table 4.3 An Example of Risks in the Green Zone

<i>Risk</i>	<i>Probability</i>	<i>Impact</i>
R1	0.10	€500.00
R2	0.10	€1000.00
R3	0.20	€400.00
R4	0.15	€1000.00
R5	0.10	€200.00

EXAMPLE 4.1

Suppose we have a project with five risks in the green zone, for which we have managed to quantify probability and impact as described in Table 4.3. The expected monetary value of the risks is computed as follows:

$$EMV = \sum_{i=1}^n p_i * c_i$$

Substituting the data in Table 4.3, we get

$$EMV = 0.10 * 500 + 0.10 * 1000 + 0.20 * 400 + 0.15 * 1000 \\ + 0.10 * 200 = 400$$

which is the amount we can allocate to manage risks in the green area. Note that the amount allows us to cover R3 and R5 and most of the costs of R1. If R1, R2, or R3 will occur in the project, we will need to find the money from another source.

A more cautious approach in the example we have made could allocate e1000, enough to cover R2 or R4 and any other risk. However, given the probabilities, all the risks in the table are far more likely not to occur than to occur and the money reserved for these risks is more likely not to be used than to be used. Thus, like in many similar situations, the techniques provide a good reference framework, but the project manager always has the responsibility of understanding the situation and context and making the appropriate choice based on all these information.

Risk monitoring and control during project execution involves three main activities:

1. **Review and Share:** Risks are regularly assessed, and their status is updated (e.g., "active," "closed," or "occurred"). These statuses are shared with stakeholders using alert levels (e.g., "green," "amber," "red").
2. **Apply Contingency Plans:** When a risk occurs, contingency plans are activated to address it. The procedures may vary in formality, and necessary resources, including budget, are allocated. Contingency plans should follow the predefined strategies from the risk planning phase to ensure proper action and communication.
3. **Revise and Iterate:** New risks may emerge as the project progresses. The process of identifying and managing risks is ongoing, with the monitoring and control process being updated as new risks or opportunities arise.

Quality Management

Quality Management in Software Development

Quality management in software development is a crucial aspect that ensures software products meet the necessary standards, requirements, and user expectations. It encompasses all the activities and processes needed to monitor, measure, and improve the quality of software throughout its lifecycle. Below are the key components and practices involved in software quality management:

A good SQA process is composed of three steps:

- **Quality planning:** which identifies the relevant standard and practices and the way to implement them
- **Quality assurance:** which focuses on ensuring that the project applies and follows the quality standards identified at the previous step
- **Quality control:** which ensures that the products respect the quality standards identified during the planning phase.

Quality Planning:

- Identifies relevant quality standards and practices.
- Defines the methods and processes to implement the standards.
- Sets quality objectives, scope, and guidelines for the project.
- Determines the tools, techniques, and metrics to be used.
- Ensures alignment with industry regulations (e.g., safety, reliability).

Quality Assurance (QA):

- Focuses on ensuring the software development process adheres to the planned quality standards.
- Involves proactive activities like inspections, code reviews, and audits.
- Ensures that the team follows best practices and process improvements.
- Emphasizes defect prevention and continuous improvement.
- May include team training and fostering a quality-driven culture.

Quality Control (QC):

- Ensures the software product meets the quality standards set during planning.
- Involves testing and measuring the product against quality criteria.
- Detects defects and deviations, and ensures corrective actions are taken.
- Includes activities such as functional testing, performance testing, and defect tracking.
- Provides validation that the final product is ready for deployment.

*****QA is about improving and ensuring the quality of the development process, while QC is about ensuring the final product meets quality standards.**

Quality Metrics:

are quantitative measures used to evaluate the quality of software products and processes. These metrics help ensure that the project meets the desired standards and objectives, and they provide insights into areas for improvement. Common quality metrics in software project management include:

Product Quality Metrics

These metrics evaluate the quality of the software product itself.

- **Defect Density:** Measures the number of defects per unit size (e.g., per thousand lines of code).
- **Mean Time to Failure (MTTF):** Measures the average time the system operates without failure.
- **Reliability:** Assesses the probability of the software performing its intended functions without failure over a specified time.
- **Code Coverage:** Evaluates the percentage of the codebase tested during testing phases (e.g., statement, branch, or path coverage).
- **Customer Reported Defects:** Tracks defects reported by end-users post-deployment.

Process Quality Metrics

These metrics measure the efficiency and effectiveness of the processes used in software development.

- **Defect Removal Efficiency (DRE):** Measures the effectiveness of the defect detection process.
- **Schedule Variance (SV):** Compares the planned schedule to the actual schedule.
- **Cost Variance (CV):** Compares the actual costs to the planned budget.
- **Review Effectiveness:** Measures the percentage of defects found during reviews versus those found later.

Project Quality Metrics

These focus on the overall health of the project.

- **On-Time Delivery Rate:** Evaluates the percentage of projects delivered on or before the deadline.
- **Effort Variance:** Measures the difference between estimated and actual effort required for the project.
- **Stakeholder Satisfaction:** Gauges how well stakeholder expectations were met.

Code Quality Metrics

These metrics assess the quality of the codebase for maintainability, readability, and performance.

- **Cyclomatic Complexity:** Measures the complexity of the program's control flow. Assesses algorithm complexity by counting loops and decision points, indicating testability and potential states.
- **Technical Debt:** Estimates the cost of fixing issues in the codebase that may delay future development.
- **Maintainability Index:** Quantifies how maintainable the software is based on complexity, lines of code, and comments.

User Quality Metrics

These metrics capture the user experience and satisfaction.

- **Customer Satisfaction Index (CSI):** Measures user satisfaction through surveys or feedback.
- **Net Promoter Score (NPS):** Assesses the likelihood of users recommending the software to others.
- **Response Time:** Measures how quickly the software responds to user actions or requests.
- **Usability Metrics:** Evaluate the ease of use, learning curve, and user error rate.